

102 - Web Development and Design Assignment

1. What is the Document Object Model (DOM)?

- DOM is a programming interface for HTML / XML documents.
- It represents the page as a tree structure where each element is a node.
- JavaScript can access and modify elements, attributes, and styles using DOM.
- Example:

```
document.getElementById("title").innerHTML = "Hello";
```

2. How do you add an event handler to a button click in JavaScript?

- Event handlers are functions executed when an event occurs (like click).
- Can be added in three ways:
 - a. inline $\rightarrow$ `<button onclick="MyFunc()">Click </button>`
 - b. Using onclick property $\rightarrow$
`document.getElementById("btn").onclick = myFunc;`
 - c. Using addEventListener (preferred) $\rightarrow$
`document.getElementById("btn").addEventListener("click", myFunc);`

3. Explain in detail how JSX works in React.

- JSX (JavaScript XML) lets us write HTML-like syntax in JS code.
- It is compiled to JavaScript using `React.createElement`.
- Advantages: easier to read, integrates UI + logic, error detection at compile-time.
- Example:

```
function Greeting() {  
  return <h1>Hello React </h1>;  
}
```

4. Discuss `useState` and `useEffect` hooks in React.

- `useState`:

- Used to store and update values inside a functional Component
- Example:

```
const [count, setCount] = useState(0);
```

- `useEffect`:

- Used to perform side effects (fetching API, timers, updating title).
- Example:

```
useEffect(() => { document.title = `Count : ${count}`; }, [count]);
```

- Together: `useState` manages state; `useEffect` reacts to changes.

5. Explain nested routes in React Router:

- Nested routes allow child routes inside parent routes.
- Useful for dashboards, profiles, or layouts with sub-pages.
- Example:

```
<Route path = "dashboard" element = {<Dashboard />}>  
  <Route path = "profile" element = {<Profile />}>  
    <Route path = "settings" element = {<Settings />}>  
  </Route>  
</Route>
```

- Advantage: Better structured & modular routing.

6. Discuss Redux architecture:

- Components of Redux:

- a. Store → Holds the entire application state.

- b. Action → Object describing events (`{type: "ADD_ITEM"}`)

- c. Reducers → Pure functions that update state based on actions.

- Flow of Data:

Component → Dispatch (Action) → Reducer → Store Updates → UI re-renders

7. Connecting Redux using hooks.

- `useSelector`: Reads data from Redux store.

- `useDispatch`: Sends actions to reducer.

- Example:

```
const count = useSelector(state => state.count);  
const dispatch = useDispatch();  
dispatch({type: "INCREMENT"});
```

8. Compare Redux with Context API & MobX.

- Redux: Predictable, strict rules, best for large apps, but verbose.

- Context API: Simple, built into React, good for small apps, but causes re-renders.

- MobX: Uses observables, less boilerplate, more flexible than Redux.

9. Role of Middleware in Redux.

- Middleware sits between dispatching an action and reducer execution.
- Handles side effects (logging, async API calls).
- Example: Redux Thunk (for async actions), Redux Saga.

10. Creating custom middleware in Redux.

- Custom middleware is a function that takes `store` $\rightarrow$ `next` $\rightarrow$ `action`.
- Example (logger middleware):

```
const logger = store  $\Rightarrow$  next  $\Rightarrow$  action  $\Rightarrow$  {  
  console.log("Action:", action);  
  return next(action);  
}
```
- Helps in debugging and handling async operations.